

FASCICULE DE PHYSIQUE/CHIMIE DE PT*

*

CAPACITÉS NUMÉRIQUES

Banque PT

*

BAPTISTE PORTELLI

Chapitre 1

Electronique

L'idée forte de ce chapitre sera de comprendre l'action des filtres sur des signaux périodiques non-sinusoidaux, typiquement des signaux créneaux ou triangulaires.

- Dans un premier temps, le but est de créer numériquement le signal d'entrée, à partir de sa décomposition en série de Fourier, laquelle sera fournie dans un énoncé de concours¹ ;
- Dans un second temps, on pourra s'attacher aux propriétés spécifiques du filtre lorsque celui-ci est alimenté en régime sinusoïdal forcé. L'objectif sera de savoir tracer un diagramme de Bode avec Python ;
- Dans un dernier temps, le but sera de pouvoir construire la réponse du filtre au signal périodique d'entrée préalablement préparé.

1.1 Synthèse d'un signal créneau à partir de sa DSF

Avertissement : cet exercice - très simple - constitue un préalable pour le reste de la planche.

On rappelle qu'une fonction périodique $e(t)$ de période T et de valeur $\langle e \rangle$ se décompose en série de Fourier comme :

$$e(t) = \langle e \rangle + \sum_{n=1}^{+\infty} c_n \cos(\omega_n t + \phi_n) \quad \omega_n = n\omega \quad \text{et} \quad \omega = \frac{2\pi}{T}$$

On étudie, ici, un signal créneau $e(t)$ de période T , de valeur moyenne $\langle e \rangle$ et d'amplitude E .

Dans le cas où l'on fixe l'origine des temps de telle manière que $e(t) - \langle e \rangle$ soit une fonction impaire de t , on peut montrer que la décomposition en série de Fourier de $e(t)$ s'écrit :

$$e(t) = \langle e \rangle + \sum_{p=0}^{+\infty} c_{2p+1} \sin(\omega_{2p+1} t) \quad \text{où} \quad \omega_{2p+1} = (2p+1)\omega \quad , \quad \omega = 2\pi/T \quad \text{et} \quad c_{2p+1} = \frac{4E}{(2p+1)\pi}$$

On donne ci-dessous le script Python permettant de tracer un tel signal.

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 # -----
5
6 fe = à compléter # fréquence du signal d'entrée en Hz
7 E= à compléter # amplitude du signal créneau
8 moyenne = à compléter #moyenne du signal créneau
9
```

1. Le calcul des coefficients de Fourier est strictement hors-programme des CPGE en physique.

```

10 nmax=500 # Nombre d'harmoniques prises en compte
11
12 # -----
13 # A. Synthèse spectrale du signal d'entrée - Tracé de e(t)
14 # -----
15
16 cn=[] #Définit la liste des coefficients de Fourier
17 phin=[] #phases
18
19
20 for i in range(nmax):
21     cn.append(4*E/((2*i+1)*np.pi)) # Amplitudes des différentes harmoniques
22     phin= # A compléter
23
24
25 # Tracé de la fonction e(t) à partir de la DSF - Représentation temporelle -
26
27
28 def e(t):
29     val=moyenne # Valeur moyenne
30     for n in range(0, len(cn)):
31         #val += cn[n]*np.cos(2*np.pi*n*fe*t+phin[n])
32         val += cn[n]*np.cos(2*np.pi*(2*n+1)*fe*t+phin[n])
33     return val
34
35 t=np.linspace(0,2/fe,500) # Affiche la fenetre temporelle, (ici deux périodes) et le niveau
36 #de discrétisation (ici 500 points régulièrement espacés).
37 plt.plot(t,e(t), 'b-', label=r'$e(t)$')
38 plt.xlim(0,2/fe), plt.xlabel(r"$t$ (en s)")
39 #plt.ylim(-0.5,2.5)
40 plt.ylabel(r"$e(t)$ (en V)")
41 plt.legend()
42 plt.grid()
43 plt.savefig('fig_entree.jpg') # Exporte le fichier au format jpg.
44 plt.show()

```

1. Tester le code dans le cas d'un signal de fréquence $f_e = 1,0 \text{ kHz}$, d'amplitude $E = 1 \text{ V}$ et de valeur moyenne $\langle e \rangle = 1 \text{ V}$. On pourra faire plusieurs tests en modifiant un paramètre et en laissant les autres inchangés.
2. $e(t)$ s'identifie à sa décomposition en série de Fourier uniquement dans le cas où on fait tendre le nombre d'harmoniques n_{max} vers $+\infty$. Etudier l'influence de n_{max} sur la construction progressive de la fonction créneau. Augmenter progressivement n_{max} comme suit :

$$n_{max} = 1, 2, 3, 5, 50, 100, 500$$

3. Adapter le code pour représenter un signal triangulaire $e'(t)$ de fréquence $f' = 2 \text{ kHz}$, d'amplitude $E' = 0,5 \text{ V}$ et de valeur moyenne $\langle e' \rangle = 0,75 \text{ V}$.

Pour cela, on rappelle l'expression de la décomposition en série de Fourier d'un signal triangulaire, dans le cas où l'origine des temps a été choisie pour que $e'(t) - \langle e' \rangle$ soit pair :

$$e'(t) = \langle e' \rangle - \sum_{p=0}^{\infty} \frac{8E'}{(2p+1)^2\pi^2} \cos(\omega_{2p+1}t) \quad (1.1)$$

Comparer l'effet de n_{max} dans la construction de $e(t)$ et de $e'(t)$. On affichera simultanément $e(t)$ et $e'(t)$ en veillant à proposer une légende.

1.2 Filtrage d'une tension périodique par un filtre d'ordre 1

1.2.1 Filtrage d'une tension créneau par un filtre passe-bas d'ordre 1

Le but de cet exercice est d'étudier l'action d'un filtre d'ordre 1 sur le signal créneau défini dans l'exercice 1.1.

1. Rappeler la fonction de transfert canonique $\underline{H}(j\omega)$ d'un filtre passe-bas d'ordre 1, de gain statique $\mathcal{H}_0$ et de pulsation de coupure ω_c .
2. Code Python associé au tracé du diagramme de Bode :

```

1
2 H0=1
3 G0=np.abs(H0)
4
5
6 # Tracé du diagramme de Bode du filtre d'ordre 1
7
8
9 def G1(f,fc):
10     return G0/(1+(f/fc)**2)**0.5
11
12 def Phi1(f,fc):
13     return -np.arctan(f/fc) if H0>0 else np.pi-np.arctan(f/fc)
14
15
16
17 def GdB(f,fc): # Fonction gain en dB
18     return 20*np.log10(G1(f,fc))
19
20
21
22 fmin=0.0
23 fmax=10**6
24 points=10**5
25
26
27 f=np.linspace(fmin,fmax,points) #
28
29
30 # TRACE DE LA COURBE DE GAIN
31
32 fig, ax=plt.subplots()
33 ax.grid(which='minor')
34
35
36 fc=1000 # Fixe la valeur de fc en Hz
37
38
39 plt.plot(f, GdB(f,fc), c='r', label=r'$G_{dB}$')
40 plt.xlabel(r'$f$')
41 plt.ylabel(r'$G_{dB}(f)$ en dB')
42 plt.xscale('log') # Echelle logarithmique sur les fréquences
43 plt.legend()
44 plt.xlim(1,10**5) # Domaine de variation de la fréquence
45 plt.ylim(-40,5)
46 plt.grid(True)
47 plt.savefig('GdB_passe_bas1.jpg')
48 plt.show()

```

- (a) Justifier les lignes 10 et 13 du script.
 - (b) Vérifier le bon fonctionnement du code et tracer $G_{dB}(f)$.
 - (c) Tracer, de même, la courbe de déphasage $\varphi(f)$.
3. Code Python décrivant l'action du filtre sur le signal créneau :

```

1
2 # Action du filtre passe bas d'ordre 1 sur le signal créneau
3
4 fc=0.1*fe # Choix de la fréquence de coupure
5
6
7 def s(t,fc) : # Définit la fonction en sortie du filtre
8     val = G1(0,fc)*moyenne

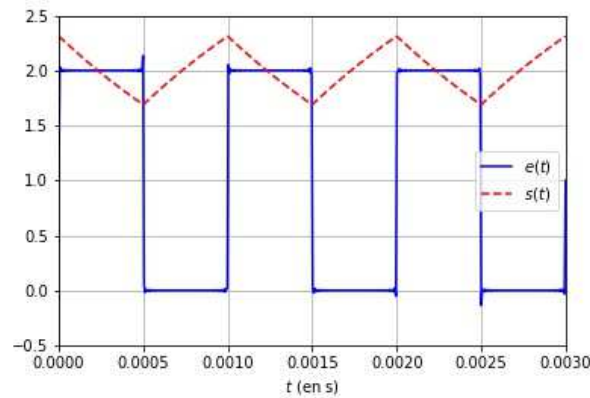
```

```

9      for n in range(0, len(cn)):
10         an = G1((2*n+1)*fe, fc)*cn[n]
11         psin=phin[n]+Phi1((2*n+1)*fe, fc)
12         val += an*np.cos(2*np.pi*(2*n+1)*fe*t + psin)
13     return val
14
15
16     plt.plot(t,e(t), 'b-', label=r'$e(t)$')
17     plt.plot(t,s(t,fc), linestyle='dashed', c='r', label=r'$s(t)$')
18     plt.xlim(0,3/fe), plt.xlabel(r"$t$ (en s)")
19     plt.ylim(-0.5,2.5)
20     plt.legend()
21     plt.grid()
22     plt.show()

```

- Commenter les lignes 8, 10 et 11 de cette portion de code.
- Vérifier le bon fonctionnement du code.
- Etudier l'impact de la fréquence de coupure f_c du filtre sur $s(t)$. Commenter le comportement du filtre dans le cas où $f_c = 0,1 f_e$.
- Que vaut $\mathcal{H}_0$ dans le cas ci-dessous ?



1.2.2 Application - Filtrage d'une tension triangulaire par un filtre passe-haut d'ordre 1

En vous inspirant de l'étude conduite en 1.2.1, réaliser les consignes suivantes (dans l'ordre que vous souhaitez) :

- Ecrire un code Python permettant de simuler l'action d'un filtre passe-haut d'ordre 1 sur un signal triangulaire d'amplitude $E = 1 V$, de fréquence fondamentale $f = 1 kHz$ et de valeur moyenne $\langle e \rangle = 1 V$. On choisira un filtre tel que $\mathcal{H}_0 = 5$.

On rappelle que la décomposition en série de Fourier d'un signal triangulaire est donné par :

$$e(t) = \langle e \rangle - \sum_{p=0}^{\infty} \frac{8E}{(2p+1)^2\pi^2} \cos(\omega_{2p+1}t) \quad \text{avec} \quad \omega_{2p+1} = (2p+1)\omega \quad (1.2)$$

- Afficher le diagramme de Bode du filtre (en gain et en déphasage).
- Modifier la fréquence de coupure f_c et analyser la réponse du filtre selon que $f \ll f_e$ ou $f \gg f_e$.

Chapitre 2

Thermodynamique

2.1 Évolution de la température d'une pièce climatisée

Le but de cet exercice est de tracer numériquement à l'aide d'un code Python l'évolution de la température $T(t)$ d'une pièce climatisée par un climatiseur de puissance électrique P .

Données :

- Dimensions de la pièce : $L_x = 10\text{ m}$, $L_y = 7\text{ m}$ et $L_z = 3\text{ m}$
- Température initiale de la pièce $T_0 = 25^\circ\text{C}$;
- Température de l'extérieur $T_{ext} = 35^\circ\text{C}$. Cette température reste constante dans tout le problème.
- On néglige les fuites thermiques.
- L'air de la pièce est assimilé à un gaz parfait de coefficient $\gamma = 1,4$ indépendant de T .

On suppose qu'à chaque instant, l'efficacité $e(t)$ du climatiseur s'exprime comme $e(t) = \alpha \times e_c(t)$ où $e_c(t)$ est l'efficacité instantanée de Carnot.

1. **Travail théorique** : Montrer que l'équation différentielle vérifiée par la température de la pièce s'écrit :

$$\frac{dT}{dt} = \frac{\alpha P}{C_v} \frac{T}{T - T_{ext}}$$

où C_v est la capacité thermique totale du gaz à volume constant.

2. **Travail numérique** : Faire un code Python permettant de tracer :

- (a) $T(t)$ pour $P = 1,0\text{ kW}$, $P = 1,5\text{ kW}$ et $P = 2\text{ kW}$. Dans cette question, on fixera $\alpha = 0.3$. Comme intervalle de temps, on choisira $\Delta t = [0, 150]\text{ s}$. On tracera les trois courbes sur un même graphe. Commenter.
- (b) $T(t)$ pour $\alpha = \{0,3, 0,5, 1\}$ pour $P = 1,5\text{ kW}$. On tracera les trois courbes sur un même graphe. Commenter.
- (c) Afficher les valeurs numériques du nombre totale de moles n et de C_v .
- (d) Modifier le code pour simuler l'effet d'un réchauffement de la même pièce par une pompe à chaleur de même puissance.

Rappels : méthode d'Euler pour résoudre numériquement une équation différentielle :

Nous illustrons ci-dessous la méthode d'Euler pour résoudre une équation différentielle du type

$$\frac{dy}{dt} = F(y)$$

Sur l'exemple, nous avons choisi $F(y) = 1 - y$. Dans ce cas, l'équation est linéaire, ce qui permettra de comparer le résultat issu du code avec celui qu'on est à même de déterminer théoriquement.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
```

```

3
4
5 #ti : début de l'intervalle
6 #tf : fin de l'intervalle
7 #n : découpage de l'intervalle de temps en n sous-segments de même durée
8
9
10 n=100
11
12 def F(y): #F(y) est telle que dy/dt = F(y)
13     return(1-y)
14
15
16 def euler(ti,tf,y0,n) :
17     dt=(tf-ti)/n
18     t=ti
19     y=y0
20     les_t=[t] # Définit la liste des instants
21     les_y=[y] # Définit la variable physique y (dépendant de t) comme une liste.
22
23
24 #Equation du type : y'=F(y) avec y'=(y(k+1)-y(k))/dt
25
26
27
28 for k in range(n) :
29     t,y = t+dt , y+dt*F(y)
30     les_t.append(t)
31     les_y.append(y)
32
33
34
35 return (les_t , les_y)
36
37 les_t , les_y = euler(0,10,0,n) # Cas où on simule l'évolution de y(t) entre ti=0 et tf=10
38
39
40
41 plt.plot(les_t , les_y , 'r')
42 plt.legend()
43 plt.xlabel('$t$')
44 plt.ylabel('$y(t)$')
45 plt.legend()
46 plt.grid()
47 plt.show

```

2.2 Résolution numérique de l'équation de la chaleur

2.2.1 Cas 1D

Le but est déterminer l'évolution temporelle de la température $T(x,t)$ dans un conducteur thermique de longueur $L = 1\text{ m}$ soumis aux conditions suivantes :

- Pour $t < 0$, on suppose que le barreau est à l'équilibre thermique, en chacun de ces points, avec l'extérieur de température $T_0 = 20^\circ\text{C}$;
- À partir de $t = 0$, on place deux thermostats de température $T_1 = 100^\circ\text{C}$ et $T_2 = 0^\circ\text{C}$ respectivement en $x = 0$ et $x = L$.

Le but est de simuler numériquement le transfert de chaleur par conduction thermique au sein du barreau. On supposera que le transfert thermique est strictement conductif et axial. Aucune source de chaleur n'est présente dans le conducteur.

1. Démontrer l'équation de la chaleur vérifiée par $T(x,t)$ au sein du barreau conducteur. On notera D le coefficient de diffusion thermique et, dans la suite, on prendra $D = 1\text{ S.I}$ (préciser l'unité).
2. **Discrétisation de l'équation de la chaleur. Méthode d'Euler explicite**

- (a) On discrétise la longueur L et la durée d'étude notée **Temps** en intervalles réguliers de pas dx et dt . On note N_x et N_t les nombres de points permettant de décrire les intervalles $[0, L]$ et $[0, \text{Temps}]$. Exprimer dx en fonction de L et N_x d'une part et dt en fonction de T et N_t d'autre part. Compléter alors les lignes L15 et L16 du script Python.
- (b) On note T_j^n la température au point M d'abscisse $x = j \times dx$ avec $j \in [0..N_x - 1]$ et à l'instant $t = n \times dt$ avec $n \in [0..N_t - 1]$.
- (c) Montrer que la discrétisation de $\frac{\partial^2 T}{\partial x^2}$ (Laplacien 1D) s'exprime à travers une relation entre T_j^n , T_{j-1}^n , T_{j+1}^n et dx .
- (d) Justifier que la discrétisation de l'équation de la chaleur 1D conduit à la relation de récurrence :

$$T_j^{n+1} = T_j^n = \alpha \times (T_{j+1}^n + T_{j-1}^n - 2T_j^n)$$

où on exprimera α en fonction de D , dx et dt . On complètera alors la ligne L30 du script Python.

3. **Conditions aux limites** - La température est définie comme une liste **T** pouvant posséder jusqu'à N_x éléments. Quelles valeurs donner à $T[0]$ et $T[-1]$ pour qu'elles soient compatibles avec les conditions aux limites imposées? Compléter alors les lignes L26 et L27.
4. On définit **DELTA_T** comme la liste contenant les éléments ayant pour valeur $\alpha(T_{j+1}^n + T_{j-1}^n - 2T_j^n)$.
 - (a) Quelle doit-être la longueur de cette liste pour que **DELTA_T** soit définie sur l'ensemble du barreau conducteur?
 - (b) Exprimer **DELTA_T**[1, -1] comme

$$\text{DELTA_T}[1 : -1] = \alpha \times (T[a:] + T[b:] - 2.0 \times T[c:d])$$

où a , b , c et d sont à préciser. Compléter alors la ligne L37

5. Expliquer le principe de la boucle démarrant à partir de L36.
6. Après avoir fait tourné le code, le modifier dans le cas d'un conducteur thermique ayant
 - (a) une conductivité thermique deux fois plus élevée;
 - (b) une capacité thermique deux fois plus élevée.
7. (a) Sur quelle variable agir pour observer le profil de température en régime stationnaire? Modifier cette variable jusqu'à obtenir le profil stationnaire.
 - (b) Vérifier que le profil est bien en accord avec celui prévu par la théorie.
 - (c) Quel est l'effet du coefficient de diffusion sur le temps d'accès au régime stationnaire?

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  # Constantes utiles
5  # Toutes les grandeurs dans l'unité de base du SI
6  D = 1.0          # coefficient de diffusion
7  L = 1.0          # Longueur du barreau
8  Temps = 0.1      # Durée d'intégration
9  Nx = 100         # Nombre de points sur le barreau
10 Nt = int(1e5)     # Nombre d'instant
11 T1 = 100.0        # Température en x=0
12 T2 = 0.0          # Température en x=L
13 T0 = 20.0         # Température initiale
14 N_courbes = 10    # Nombres de courbes souhaitées
15 dx = ? à compléter # pas de la grille (espace)
16 dt = ? à compléter # pas de temps
17
18 # Initialisation
19 x = np.linspace(0.0, L, Nx)
20
21
22 # Condition initiale sur la température
23 T = np.ones(Nx) * T0
24
25 # Conditions aux limites

```

```

26 T[0] = ? à compléter
27 T[-1] = ? à compléter
28
29 #
30 alpha = ? (à compléter)
31 print("La valeur de alpha est {0:4.2e}".format(alpha))
32
33 DELTA_T = np.zeros((Nx))
34 plt.figure()
35
36 for n in range(Nt):
37     DELTA_T[1:-1] = ? à compléter
38     T += DELTA_T
39
40
41     if n in int(Nt/N_courbes)*np.arange(N_courbes):
42         print(n)
43         plotlabel = "t={0:4.2f}s".format(n*dt)
44         plt.plot(x,T, label=plotlabel)
45 plt.xlabel(u'$x$/m')
46 plt.ylabel(u'$T$/K')
47 plt.title(u'Equation de la chaleur 1D')
48 plt.legend()
49 plt.savefig("Diffusion-chaleur-1D.pdf")
50 plt.show()

```

2.2.2 Cas 2D

On considère une plaque plane rectangulaire de longueur $L_x = 1\text{ m}$ et de largeur $L_y = 0,5\text{ m}$ soumise aux conditions aux limites suivantes :

- $T(0, y, t) = T_1 = 100^\circ\text{C}$ et $T(L_x, y, t) = T_2 = 80^\circ\text{C}$;
- $T(x, 0, t) = T(x, L_y, t) = T_e = 10^\circ\text{C}$.

Initialement, la température de la plaque est uniforme et vaut $T_0 = 30^\circ\text{C}$ pour chaque point M de la plaque.

```

1
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Constantes utiles
6 # Toutes les grandeurs dans l'unité de base du SI
7 D = 1.0          # coefficient de diffusion
8 Lx = 1.0         # Longueur du barreau
9 Ly = 0.5         # Largeur du barreau
10 Temps = 0.05    # Durée d'intégration
11 Nx, Ny = 100, 50 # Nombre de points sur le barreau
12 Nt = int(1e5)    # Nombre d'instant
13 T1 = 100.0       # Température en x=0
14 T2 = 80.0        # Température en x=Lx
15 T0 = 30.0        # Température initiale
16 Te = 10.0        # Température extérieure
17 N_courbes = 5    # Nombres de courbes souhaitées
18 dx = ?          # pas de la grille (espace)
19 dy = ?
20 dt = ?          # pas de temps
21
22 # Pour la figure
23 x = np.linspace(0, Lx, Nx)
24 y = np.linspace(0, Ly, Ny)
25 X, Y = np.meshgrid(x, y, indexing='ij')
26
27 # Condition initiale
28 T = T0*np.ones(shape=(Nx, Ny))
29 # Conditions aux limites

```

```

30 CL1 ?
31 CL2 ?
32 CL3 ?
33 CL4 ?
34
35 alpha_x = ?
36 alpha_y = ?
37 print("La_valeur_de_alpha_x_est_{}".format(alpha_x))
38 print("La_valeur_de_alpha_y_est_{}".format(alpha_y))
39 DELTA_T = np.zeros((Nx,Ny))
40
41
42 plt.ion() # Active le mode interactif
43 plt.figure(figsize=(10*Lx,10*Ly))
44 CS = plt.pcolor(X,Y,T)
45 plt.clim(-0.1,100.1) # Créé une échelle de teintes entre -0.1 et 100.1
46 cbar = plt.colorbar(CS, shrink=0.9) # Affiche l'échelle de teinte
47 cbar.ax.set_ylabel(u'Température')
48 plt.xlabel(u'x')
49 plt.ylabel(u'y')
50 for n in range(Nt):
51     # Diffusion thermique dans le barreau
52     DELTA_T[1:-1,1:-1] = ?
53
54     # itération
55     T += DELTA_T
56
57     if n in int(Nt/N_courbes)*np.arange(N_courbes):
58         #print(n)
59         plt.pcolor(X,Y,T)
60         plt.clim(-0.1,100.1)
61         plt.title("t={0:4.3f}s".format(n*dt))
62         plt.draw()

```

1. Compléter les lignes L18 et L19 où dx et dy désignent les pas de discrétisation selon x et y .
2. Compléter, de même, la ligne L20 où dt désigne le pas de discrétisation en temps.
3. Traduire chacune des quatre conditions aux limites entre les lignes L30 et L33
4. (a) Rappeler l'équation de la chaleur vérifiée par $T(x, y, t)$ en un tout point $M(x, y)$ de la plaque.
 (b) On note $T_{i,j}^n = T(x, y, t)$ et $T_{i,j}^{n+1} = T(x, y, t + dt)$. Montrer, à l'aide de la méthode d'Euler, que l'équation de la chaleur peut être discrétisée comme suit :

$$T_{i,j}^{n+1} = T_{i,j}^n + \alpha_x \times f(T_{i+1,j}^n, T_{i-1,j}^n, T_{i,j}^n) + \alpha_y \times g(T_{i,j+1}^n, T_{i,j-1}^n, T_{i,j}^n)$$

où on identifiera les fonctions f et g et on déterminera les expressions de α_x et α_y en fonction de D , dt , dx et dy . On complètera alors les lignes L35 et L36

5. On note DELTA_T la liste des valeurs $T_{i,j}^{n+1} - T_{i,j}^n$ en chaque point $M_{i,j}$ de la grille. En vous inspirant du cas 1D, compléter la ligne L52 en exprimer DELTA_T[1:-1,1:-1].

Chapitre 3

Incertitudes de mesure

3.1 Étude d'un système de pendules couplés

```
1
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Valeurs expérimentales des grandeurs physiques
6
7 m=0.0671 # masse de la tige
8 M=0.222 #1 masse de la masselotte
9 g=9.81
10 ltige=0.50 # longueur de la tige
11 fs=0.738 # fréquence du mode symétrique
12 fa=0.767 # fréquence du mode antisymétrique
13
14
15 # Saisie des demi-étendues estimées
16
17 am = 0.001 # incertitude de 1 g
18 al = 0.002 # incertitude de 2 mm
19 af = 0.001 # incertitude de 1 mHz
20
21 # Simulation de N expériences
22
23 N=10000
24
25 # Calcul de Ccal
26
27 M_tot = []
28 L = []
29 J = []
30 C = []
31
32 for k in range(N) :
33     mt=np.random.uniform(m-am,m+am)
34     Mn=np.random.uniform(M-am,M+am)
35     Ltige=np.random.uniform(ltige-al,ltige+al)
36     Fs=np.random.uniform(fs-af,fs+af)
37     Fa=np.random.uniform(fa-af,fa+af)
38     M_tot.append(m+M)
39     L.append(ltige*(mt/2+Mn)/(Mn+mt)) # Position du centre de gravite
40     J.append(((mt+Mn)*g*ltige*(mt/2+Mn)/(Mn+mt))/(4*3.14**2*Fs**2)) # Mesure de J
41     C.append(2*3.14**2*(Fa**2-Fs**2)*((mt+Mn)*g*ltige*(mt/2+Mn)/(Mn+mt))/(4*3.14**2*Fs**2))
42
43 #Calcul de la valeur moyenne de et de l'écart-type
44
45 J_moy=sum(J)/N # Valeur moyenne
46 uJ=np.std(J) # Ecart-type
47
48 C_moy=sum(C)/N # Valeur moyenne
```

```

49 uC=np.std(C) # Ecart-type
50
51
52 Mtot_moy=sum(M_tot)/N # Valeur moyenne
53 uMtot=np.std(M_tot) # Ecart-type
54
55 L_moy=sum(L)/N # Valeur moyenne
56 uL=np.std(L) # Ecart-type
57
58
59
60
61 # Affichage des résultats
62
63 print(f' Meilleur estimateur <J>: {J_moy} kg.m^2 ')
64 print(f' Incertitude-type u(J): {uJ} kg.m^2 ')
65
66 print(f' Meilleur estimateur <C>: {C_moy} N.m ')
67 print(f' Incertitude-type u(C): {uC} N.m ')
68
69
70
71 #print(f' Meilleur estimateur <M_tot>: {Mtot_moy} kg )
72 #print(f' Incertitude-type u(J): {uMtot} kg ')
73 print(f' Meilleur estimateur <L>: {L_moy} m ')
74 print(f' Incertitude-type u(L): {uL} m ')
75
76 #Tracé de l'histogramme
77
78 x=np.linspace(np.min(J),np.max(J),5000)
79 Gaussienne = [1/(np.sqrt(2*np.pi*uJ**2))*np.exp(-0.5*((z-J_moy)**2/(uJ**2)))] for z in x]
80 plt.hist(J, bins='rice', density=True, label=" Valeurs simulées ")
81 plt.plot(x,Gaussienne, 'k-', label = "Gaussienne associée")
82 plt.title(f' Simulation de {N} mesures ')
83 plt.xlabel(' Mesure de $J_{\Delta}$ ')
84 plt.legend()
85 plt.show()

```

3.2 Optique

3.2.1 TP - Caliens : mesure de la largeur d'une fente

Le logiciel Caliens utilisé en TP permet de mesurer l'intensité lumineuse d'une figure d'interférences ou de diffraction.

On éclaire une fente fine de la largeur a , et située à une distance D de la barrette CCD du capteur, avec un Laser de longueur d'onde $\lambda = 650 \text{ nm}$ supposée connue exactement.

L'acquisition de la figure de diffraction avec Caliens permet de remonter à la valeur de la largeur a de la fente.

Pour cela, on admet que l'intensité diffractée sous un angle θ s'exprime comme :

$$I(u) = I_0 \text{sinc}^2(u) \quad \text{où} \quad u \stackrel{\text{def}}{=} \frac{\pi \sin \theta a}{\lambda}$$

1. Déterminer la largeur L de la tache centrale de diffraction. On se placera dans les conditions de Gauss et on exprimera le résultat en fonction de a , λ et D .
2. Les éléments du dispositif (Laser polarisé, polariseur, pupille diffractante, capteur CCD) sont disposés sur un banc optique gradué au mm . On note x_1 et x_2 les valeurs correspondant aux positions de la pupille diffractante d'une part et du capteur CCD d'autre part.
 - On mesure $x_1 = 63,1 \text{ cm}$ et $x_2 = 10,8 \text{ cm}$ (l'étudiant pourra remplacer par les valeurs qu'il a obtenues expérimentalement en TP) ;

- En utilisant la fonction **Curseurs** du logiciel Caliens, on repère les deux extrémités de la tâche centrale. On note $X_1 = 12,17\text{ mm}$ et $X_2 = 16,33\text{ mm}$.

En utilisant la méthode de Monte-Carlo¹

- (a) Déterminer la valeur moyenne $\langle a \rangle$ et l'incertitude type $u(a)$ de la largeur a de la fente.
- (b) Écrire le résultat de mesure sous une forme canonique.

3.2.2 TP - réseaux : mesure du pas d'un réseau

Un spectroscope à réseau utilise un goniomètre qui est un appareil de mesure d'angle précis à $1'$ d'angle près. En positionnant le réticule sur la raie au minimum de déviation, on montre que, dans l'ordre 1

$$2 \sin \frac{|\alpha_1 - \alpha_{-1}|}{4} = \frac{\lambda}{a}$$

où α_1 et α_{-1} sont les angles indiqués sur le vernier. On choisit, comme longueur d'onde de référence, la raie verte du mercure à $\lambda = 546,1\text{ nm}$. On mesure respectivement, $\alpha_1 = 109^\circ + 5'$ et $\alpha_{-1} = 71^\circ + 2'$.

1. En utilisant la méthode de Monte-Carlo, déterminer $\langle a \rangle$ et l'incertitude type $u(a)$.
2. Le constructeur indique que le réseau possède 600 traits par mm . Commenter.

1. Vous pourrez vous inspirer du code relatif à l'étude des pendules couplés.

Table des matières

1	Electronique	3
1.1	Synthèse d'un signal créneau à partir de sa DSF	3
1.2	Filtrage d'une tension périodique par un filtre d'ordre 1	4
1.2.1	Filtrage d'une tension créneau par un filtre passe-bas d'ordre 1	4
1.2.2	Application - Filtrage d'une tension triangulaire par un filtre passe-haut d'ordre 1	6
2	Thermodynamique	7
2.1	Évolution de la température d'une pièce climatisée	7
2.2	Résolution numérique de l'équation de la chaleur	8
2.2.1	Cas 1D	8
2.2.2	Cas 2D	10
3	Incertitudes de mesure	13
3.1	Étude d'un système de pendules couplés	13
3.2	Optique	14
3.2.1	TP - Caliens : mesure de la largeur d'une fente	14
3.2.2	TP - réseaux : mesure du pas d'un réseau	15